

Rob) 1. Instead of multiplication, the ALU instruction was "sal \$2, %eax". "sal" stands for Shift Arithmetic Left Long. According to Bryant and O'Hallaron (2016), the sal instruction performs a left shift which is a standard binary operation. Shifting left by k bits is an equivalent to 2^k , thus the compiler uses a 2-bit shift to multiply by 4 (2^2).

Source: https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/books/CSAPP_2016.pdf

Rob) 2. Pseudocode

Push	r_rbp	// Save caller's base pointer
Copy	r_rsp, r_rbp	// Establish new stack frame base
Store	r_edi, [r_rbp - 4]	// Store input x in local stack space
Load	[r_rbp - 4], r_edx	// r_edx = x, Base value for additions
Copy	r_edx, r_eax	// r_eax = x, the accumulator
Shift_left	2, r_eax	// r_eax = r_eax << 2. $x * 2^2 = x * 4$.
Add	r_edx, r_eax	// r_eax = 4x + x = 5x
Shift_left	3, r_eax	// r_eax = 5x << 3. $5x * 2^3 = 40x$.
Add	r_edx, r_eax	// r_eax = 40x + x = 41x
Pop	r_rbp	// Restore caller's base pointer
Return		// x * 41 in r_eax

Justin) 3.

1. After x is filled with the argument passed into it from register %edi, the program copies the contents of variable x into registers %edx and %eax (so %edx = x and %eax = x)
2. Next, it arithmetically shifts the contents of %eax six times to the left, then storing the result in %eax (meaning %eax = $2^6(x) = 64x$)
3. Lastly, it subtracts the contents of %eax by the contents of %edx and stores the result in %eax (%eax - %edx = $64x - x = 63x$, which is now stored in %eax)
4. As %eax is the register that holds the return value, that means the function is confirmed to have returned $x * 63$

Shai) 4. Pseudocode

```
// Push RBP into stack frame
push    r_rbp

// Copy contents of RSP into RBP
store   r_rsp, r_rbp
```

```

// Initialize registers (x = input)
store      r_edi, r_rbp - 4      // RBP-4 = x
store      rbp - 4, r_edx        // EDX = x
store      r_edx, r_eax          // EAX = x

// Compute results
shift_left 2, r_eax              // EAX << 2 = 4x
add         r_edx, r_eax         // EAX + EDX = 5x
negate      r_eax                // -EAX = -5x

// Pops top of stack into RBP / Returns -5x
pop         r_rbp

```

Justin) 5.

This time, instead of multiplying indirectly through a mix of left shifts, additions, subtractions, and negations, it just uses `imull` to return the intended product of $x * 61$.

The cause for this lies on the compilers, of which they do their best to optimize the code as much as possible. The instruction `imull` at base actually takes longer than a shift and a subtract/add when it comes to multiplying by a constant that is a power of 2 (done simply through an appropriate bitshift, an instruction needing one cycle for completion no matter the number of shifts), or in some cases multiplying by other constants that aren't powers of two yet can be reached by dealing with powers of 2 with further manipulation via adding and subtracting by 1 (each kind of operation involved here needing one cycle each), both of which can not only do the job faster for some constants, but also make it less resource intensive than the `imull` instruction.

Despite shifts and adds/subtracts seemingly being a faster course of action, sometimes `imull` is utilized instead because the constant involved cannot be easily accessed through powers of 2, not without a longer sequence of shifts and adds/subtracts that end up taking longer than `imull`.

Sources:

<https://stackoverflow.com/questions/9083743/is-bit-shifting-o1-or-on#:~:text=You%20may%20think%20shifting%208,17>

<https://medium.com/@akumar54/how-compilers-optimize-multiplication-a-deep-dive-with-assembly-and-debugging-271a3aaa448d>

[https://www.phys.ufl.edu/~coldwell/MultiplePrecision/fpvsintmult.htm#:~:text=Central%20Processing%20Unit%20\(CPU\)%3A,bit%20integer%20multiply%3A%2020%20cycles](https://www.phys.ufl.edu/~coldwell/MultiplePrecision/fpvsintmult.htm#:~:text=Central%20Processing%20Unit%20(CPU)%3A,bit%20integer%20multiply%3A%2020%20cycles)

Shai) 6.

The -O flag (denoting **O**ptimize as hinted) used by the GNU Assembler is intended to optimize assembly code for runtime and size by generating fewer ALU instructions and/or finding substitutes that aim to emulate the original code. This is evident when comparing both the Lab4.s and Lab4_O.s files, as the one compiled without the -O flag has noticeably more lines (130 lines including whitespaces) vs the opposite which has less (80 including whitespaces). Going further, there are more stark differences between the two even in the first dummy function ($x * 4$): the unoptimized assembly code goes through the whole process, from pushing the register into the stack, the actual computations themselves, to returning them into a register, whereas the optimized one utilizes an ALU instruction that does the computations and returning the result (*leal*). The *leal 0(%rdi,4), %eax* line in the Lab4_O code specifically is meant to load the (indirect) effective address of $4 * RDI + 0$ in EAX, or in other words, it takes the contents of RDI and multiplies it to 4, adds 0, and then stores it in register EAX. The *ret* line then tells the OS to pop the return address from the stack and jump there.

Sources:

https://cs.brown.edu/courses/cs033/docs/guides/x64_cheatsheet.pdf

https://www.cs.cmu.edu/~cburch/211-sp96/gnu_devel/gpp.html

<https://web.stanford.edu/class/cs107/resources/x86-64-reference.pdf>